

IBVAP

Intelligent Border Video Analytics Platform

Smart India Hackathon 2026

Problem Statement 26187

Ministry of Home Affairs / Sashastra Seema Bal

AI-powered border surveillance system transforming standard CCTV feeds into an intelligent, tamper-evident analytics platform with sub-10ms latency.

1. Problem Statement

Border surveillance operations face critical challenges with traditional CCTV systems:

- Massive video feeds with no real-time analysis
- Manual monitoring leading to operator fatigue and missed intrusions
- No tamper-evident logging of security events
- Inability to detect and alert on spatial violations (fence crossings, zone intrusions)
- No automated recognition of persons of interest or vehicles
- Evidence vulnerable to manipulation

1.1 The Challenge

SSB (Sashastra Seema Bal) requires an intelligent platform that can:

- Process multiple camera feeds simultaneously with minimal latency
- Detect and alert on perimeter breaches in real-time
- Identify persons of interest through facial recognition
- Read and log vehicle number plates automatically
- Maintain cryptographically sealed, tamper-evident evidence logs
- Operate on commodity edge hardware without proprietary cameras

2. Solution Overview

IBVAP is a monolithic FastAPI application that ingests video from multiple sources, runs real-time AI analytics, and produces a tamper-evident audit log sealed by SHA-256 hash chaining.

2.1 Key Capabilities

- Detection & Tracking: YOLO11s with ByteTrack, 8 COCO classes, CUDA/CPU auto-fallback
- Spatial Rules: Tripwire, restricted zone, loitering, direction, night movement
- Face Recognition: SCRFD + ArcFace embeddings with cosine similarity matching
- ANPR: EasyOCR with Indian plate grammar correction and temporal consensus
- Fire/Smoke Detection: YOLOv8n fine-tuned on D-Fire dataset
- Tamper-Evident Logging: SHA-256 hash chain with Merkle checkpoints
- Multi-Source Support: RTSP, HTTP, webcam, file, YouTube Live
- Operator Dashboard: Real-time MJPEG streaming, interactive fence drawing, forensic playback

Inference latency

- < 20ms per frame (GPU), < 100ms (CPU)
- < 10ms decode-to-result
- 15 FPS analytics cadence
- Up to 8 concurrent streams with micro-batching
- Sub-millisecond per event

2.2 Per

Rule eval

Frame rat

Multi-cam

Hash cha

3. Technology Stack

Detection Model

- YOLO11s (Ultralytics) with CUDA/CPU auto-fallback
- ByteTrack (one instance per video source)
- SCRFD (InsightFace)
- ArcFace embeddings with cosine similarity
- EasyOCR with Indian plate grammar correction
- YOLOv8n fine-tuned on D-Fire dataset
- FastAPI 0.115+ with Uvicorn ASGI server
- SQLite with SQLAlchemy 2.0 ORM (WAL mode)
- SHA-256 linked list with Merkle checkpoints
- Single-file HTML/JS/CSS dashboard with MJPEG streaming
- OpenCV (cv2) with FFmpeg backend
- asyncio with threadpool for CPU-bound tasks
- Pydantic Settings with .env support
- Shapely for polygon/line intersection tests
- Docker, Docker Compose, air-gapped operation

- ultralytics>=8.3.0 (YOLO11 + ByteTrack)
 - torch>=2.2, torchvision>=0.17
 - opencv-python>=4.10.0
 - numpy>=1.26, shapely>=2.0.0, lap>=0.5.12
 - fastapi>=0.115.0, uvicorn[standard]>=0.30.0
 - sqlalchemy>=2.0.0, pydantic-settings>=2.5.0
 - insightface>=0.7.3, onnxruntime>=1.19.0
 - easyocr>=1.7.0, yt-dlp>=2024.1.1

- Tracking
- Face Detection
- Face Recognition
- ANPR / OCR
- Fire/Smoke Detection
- Backend
- Database
- Hash Chain
- Frontend
- Video Processing
- Async Processing
- Configuration
- Geometry
- Deployment

- 3.1 Python Dependencies
 -
 -
 -
 -
 -
 -
 -
 -
 -
 -

reportlab>=4.0.0, psutil>=5.9

4. System Architecture

4.1 High-Level Architecture

```
Browser (Dashboard)
| HTTP / WebSocket
v
FastAPI Application (api/main.py)
|-- CameraManager --> CameraProcessor (per camera thread)
|   |-- Video Source (RTSP/HTTP/Webcam/File)
|   |-- YOLO11 Detector (CUDA/CPU)
|   |-- ByteTrack Tracker
|   |-- Rule Engine (zones, tripwires, loiter, direction)
|   |-- Face Recognition (SCRFD + ArcFace)
|   |-- ANPR (EasyOCR + Plate Grammar)
|   |-- Fire/Smoke Detection (YOLOv8n)
|   |-- Event Manager --> Hash Chain Sealer
|-- AnalysisManager (uploaded video processing)
|-- Evidence Store (snapshots, clips, crops)
`-- SQLite Database (alerts, cameras, rules, ...)
```

4.2 Concurrency Model

Streaming is async: MJPEG generators are async def and await a new frame. Frames are encoded once per frame and shared across all viewers. Alerts are pushed via WebSocket, not polled. Inference runs in camera threads, video analysis in worker threads, and chain verification in the threadpool. Nothing expensive runs on the event loop.

4.3 Processing Pipeline

1. Capture

- Video source decoded at native resolution
- Frame resized to 640x384 analytics resolution
- Night detection, darkness scoring, enhancement
- YOLO11s inference with CUDA/CPU fallback
- ByteTrack assigns persistent IDs to detections
- Zones, tripwires, loiter, direction rules evaluated
- SCRFD detects faces, ArcFace matches against watchlist
- Plate localisation, OCR, grammar correction, consensus voting
- YOLOv8n fire and smoke detection (every 5th frame)
- Alerts written to hash chain with SHA-256 sealing
- Snapshots, clips, and crops saved to disk
- WebSocket push to connected dashboards

- 2. Resize
- 3. Scene
- 4. Object
- 5. Tracking
- 6. Rule E
- 7. Face D
- 8. ANPR
- 9. Fire/Sn
- 10. Event
- 11. Eviden
- 12. Notific

5. Spatial Rules Engine

Tripwire (Virtual Fence)

- Line segment crossing with entry/exit direction detection
- Polygon intrusion with entry, sustained presence, and exit alerts
- Dwell-time detection with boundary hysteresis and grace periods
- Wrong-direction movement across a directed line
- Sustained movement detected under low-illumination conditions

Restricted

Loitering

Direction

Night Mo

5.1 An

Every rule uses a dead band around its boundary. Within the band, a rule holds its previous conclusion instead of forming a new one. A foot point resting on a boundary therefore produces one event, not an endless alternating stream.

5.2 Debouncing

The RuleEngine enforces a cooldown per (rule, track, event), with per-event-type durations from ALERT_COOLDOWNS. This prevents alert flooding while ensuring genuine intrusions are never missed.

6. Database Models

Camera

Registered video sources (RTSP/HTTP/webcam/file)

Rule

Virtual fences, zones, loiter areas, direction rules

Alert

Tamper-evident event log with SHA-256 hash chain

Checkpoint

Periodic Merkle checkpoints sealing a range of the chain

Watchlist

Face watchlist with ArcFace embeddings

AnalysisS

Uploaded MP4 analysis runs

ANPRDet

License plate readings with evidence

FaceDete

Face detections with recognition status

AlertTriag

Operator triage actions on alerts

AuditLog

Operator action audit trail

Notificati

In-app notifications for operators

SystemS

Runtime-tunable system settings

7. API Endpoint Reference

All endpoints require X-API-Key header. Base URL: <http://localhost:8000>

GET /api/cameras

List all cameras

Register new camera

List alerts with filters

Add triage action

Upload MP4 for analysis

List plate readings

List face detections

Create rule

List incidents

Verify hash chain

List audit log entries

List notifications

List evidence media files

Per-component health check

Current threat level

Get runtime settings

Update runtime settings

List watchlist entries

Aggregate dashboard overview

MJPEG video stream

Toggle low-latency preview

POST /ap

GET /api/

POST /ap

POST /ap

GET /api/

GET /api/

POST /ap

GET /api/

GET /api/

GET /api/

GET /api/

GET /api/

GET /api/

GET /api/

GET /api/

PUT /api/

GET /api/

GET /api/

GET /stre

POST /ap

8. Security & Integrity

8.1 Authentication

All API endpoints (except /health) require the X-API-Key header. The key is validated against the configured API key in settings.

8.2 Hash Chain Integrity

Every alert is sealed into a SHA-256 hash chain. Each row stores the hash of the previous row, making any modification detectable. Periodic Merkle checkpoints allow verification of log ranges without the full database.

8.3 Evidence Protection

All evidence crops are SHA-256 hashed and the digest stored alongside the file path. This detects silent replacement of evidence images.

8.4 Soft Delete

Cameras are soft-deleted (archived) rather than hard-deleted to preserve the integrity of the hash chain.

9. Deployment

Docker

- Dockerfile + docker-compose.yml included
- No external API calls; all models run locally
- Runs on commodity CPU; GPU optional for acceleration
- All settings via .env file; no code changes needed
- `python -m uvicorn api.main:app --host 0.0.0.0 --port 8000`

Air-gapped
Edge hardware
Configurable
Startup

10. Project Structure

```

SIH26187-main/
|-- api/
|   |-- main.py           # FastAPI application (388 routes)
|   |-- schemas.py        # Pydantic request/response models
|
|-- core/
|   |-- analytics.py       # Frame analysis pipeline
|   |-- camera.py          # Camera manager and processor threads
|   |-- config.py          # All settings (Pydantic BaseSettings)
|   |-- database.py        # SQLAlchemy engine and session
|   |-- events.py          # Event manager with hash chain sealing
|   |-- hashchain.py       # SHA-256 hash chain and Merkle checkpoints
|   |-- models.py          # SQLAlchemy ORM models (12 tables)
|   |-- video_source.py    # Live and file video source handlers
|   |-- ...
|
|-- cv/
|   |-- anpr.py            # ANPR processor (EasyOCR + plate grammar)
|   |-- detector.py        # YOLO11 detector wrapper
|   |-- face.py            # Face detection and recognition
|   |-- fire_smoke.py      # Fire and smoke detection (YOLOv8n)
|   |-- rules.py           # Rule engine (zones, tripwires, loiter, direction)
|   |-- ...
|
|-- dashboard/
|   |-- index.html         # Single-file dashboard (HTML + JS + CSS)
|
|-- static/
|   |-- css/style.css      # Dashboard styles
|   |-- js/app.js          # Dashboard application logic
|
|-- tests/                 # Test suite (467 tests)
|-- docs/                  # Documentation
|-- requirements.txt        # Python dependencies
|-- Dockerfile             # Docker build file
|-- docker-compose.yml     # Docker Compose configuration
|-- alembic/               # Database migrations
|-- alerts.db              # SQLite database (runtime)

```

11. Detailed Solution Explanation

11.1 System Overview

IBVAP is built as a modular monolithic FastAPI application that transforms standard CCTV feeds into an intelligent surveillance system. The system ingests video from multiple sources simultaneously, processes each frame through a 12-stage AI pipeline, and produces cryptographically sealed, tamper-evident alerts that can be verified by third parties.

11.2 Frame Processing Pipeline

Every frame flows through the following stages:

1. CAPTURE: Video source decoded at native resolution (RTSP/HTTP/webcam/file)
2. RESIZE: Frame resized to 640x384 analytics resolution for consistent processing
3. SCENE ANALYSIS: Night detection using visual darkness scoring (not clock-based), CLAHE enhancement for low-light frames
4. OBJECT DETECTION: YOLO11s inference with CUDA/CPU auto-fallback, 8 COCO classes (person, bicycle, car, motorcycle, bus, train, truck, boat)
5. TRACKING: ByteTrack assigns persistent IDs using Kalman filters and IoU-based association
6. RULE ENGINE: Spatial rules evaluated against tracked object foot points (tripwires, zones, loitering, direction, night movement)
7. FACE DETECTION: SCRFD detects faces, ArcFace generates embeddings for watchlist matching
8. ANPR: Plate localization, EasyOCR text extraction, Indian plate grammar correction, temporal consensus voting across frames
9. FIRE/SMOKE: YOLOv8n fine-tuned on D-Fire dataset detects fire and smoke (every 5th frame)
10. EVENT SEALING: Alerts written to SHA-256 hash chain with evidence snapshots and clips
11. EVIDENCE: Snapshots, video clips, and crops saved to disk with SHA-256 integrity digests
12. NOTIFICATION: Real-time WebSocket push to connected operator dashboards

11.3 Spatial Rules Engine

The rules engine is the core intelligence of IBVAP. It uses deterministic geometric analysis to detect security violations:

TRIPWIRE (VIRTUAL FENCE): A directed line segment. When a tracked object's foot point crosses the line from one side to the other, an entry or exit alert is generated. The system uses trajectory-segment intersection (not infinite line testing) to prevent false positives.

RESTRICTED ZONE: A polygon area. When an object enters and stays for a configurable number of frames, an intrusion alert fires. Sustained presence beyond a dwell threshold generates additional alerts. Exit is only declared after a grace period to handle temporary detection loss.

LOITERING ZONE: A polygon area where objects dwelling beyond a configurable time threshold trigger alerts. Uses boundary hysteresis to prevent flapping.

DIRECTION OF TRAVEL: A directed line. Movement against the permitted direction triggers a wrong-direction alert.

NIGHT MOVEMENT: Sustained object movement detected under low-illumination conditions (visual darkness, not clock-based).

11.4 ANPR Pipeline

The ANPR system uses a multi-stage approach for accurate plate reading:

1. VEHICLE DETECTION: YOLO11s detects vehicles in the frame

2. PLATE LOCALIZATION: Edge density and contour geometry identify plate candidates within the vehicle ROI (lower 60% of vehicle box)
3. OCR: EasyOCR reads text from upscaled plate crops with multiple preprocessing variants
4. GRAMMAR CORRECTION: Indian plate grammar (LL DD LLL DDDD) corrects OCR errors using constraint satisfaction
5. TEMPORAL CONSENSUS: Per-character majority vote across multiple frames of the same tracked vehicle produces a stable, accurate reading
6. EVIDENCE: Plate crop saved with SHA-256 digest, linked to the sealed alert

11.5 Face Recognition Pipeline

1. FACE DETECTION: SCRFD detects faces in the frame
2. EMBEDDING: ArcFace generates a 512-dimensional embedding for each face
3. MATCHING: Cosine similarity against watchlist entries
4. ALERT: Match above threshold triggers a watchlist alert with the matched identity
5. EVIDENCE: Face crop saved with SHA-256 digest

12. How It Addresses the Problem

12.1 Problem-to-Solution Mapping

PROBLEM: Massive video feeds with no real-time analysis

SOLUTION: IBVAP processes up to 8 concurrent camera streams at 15 FPS with sub-10ms rule evaluation, providing real-time alerts on security violations

PROBLEM: Manual monitoring leading to operator fatigue

SOLUTION: Automated detection and alerting on fence crossings, zone intrusions, loitering, and night movement reduces the need for constant manual monitoring

PROBLEM: No tamper-evident logging of security events

SOLUTION: Every alert is sealed into a SHA-256 hash chain. Modifying any historical row invalidates all subsequent rows, making tampering detectable

PROBLEM: Inability to detect spatial violations

SOLUTION: The spatial rules engine detects tripwire crossings, zone intrusions, loitering, wrong-direction movement, and night movement with deterministic geometric analysis

PROBLEM: No automated recognition of persons or vehicles

SOLUTION: Facial recognition (SCRFD + ArcFace) identifies persons of interest against a watchlist. ANPR reads and logs vehicle number plates with Indian grammar validation

PROBLEM: Evidence vulnerable to manipulation

SOLUTION: All evidence (snapshots, clips, crops) is SHA-256 hashed and the digest stored in the hash-chained alert row, binding evidence to the log cryptographically

PROBLEM: Proprietary camera dependency

SOLUTION: IBVAP works with any standard RTSP/HTTP/webcam/file source. No proprietary cameras required

PROBLEM: High latency alerting

SOLUTION: Sub-10ms decode-to-rule evaluation and WebSocket push notifications ensure operators are alerted in real-time

12.2 Operational Benefits

- - Reduced operator workload through automated detection and alerting
 - Faster response time with real-time WebSocket notifications
 - Court-admissible evidence with cryptographic integrity verification
 - Scalable architecture supporting multiple cameras and rule types
 - Configurable thresholds and rules without code changes
 - Works on commodity edge hardware - no expensive proprietary systems
 - Air-gapped operation with no external API dependencies

13. Innovation and Uniqueness

13.1 Key Innovations

Visual Night Detection

Unlike traditional systems that use clock-based night detection, IBVAP uses visual darkness scoring based on actual frame illumination. This correctly handles floodlit areas at night and dark areas during daytime.

Temporal Consensus for ANPR

Instead of relying on a single frame's OCR output, IBVAP uses per-character majority voting across multiple frames of the same tracked vehicle. This produces stable, accurate plate readings even with motion blur and varying lighting conditions.

Indian Plate Grammar Correction

OCR errors are corrected using constraint satisfaction against the known Indian registration format (LL DD LLL DDDD). This is not a simple regex match - it enumerates all valid layouts and picks the one requiring the fewest substitutions.

SHA-256 Hash Chain Evidence

Every alert is cryptographically sealed into a hash chain. Combined with Merkle checkpoints, this allows third-party verification of log integrity without access to the full database.

Low-Latency Preview Mode

A raw frame fast path that bypasses the analytics pipeline for responsive live viewing, while analytics continues in the background. Operators get instant visual feedback without waiting for YOLO inference.

Fair OCR Scheduling

ANPR uses a fair scheduling algorithm that ensures all vehicles get read, not just the highest-scoring ones. Vehicles near frame edges get priority as they are about to leave.

Replay Suppression for Looping Files

When a video file loops, the system recognizes repeated events and suppresses duplicates while still logging genuinely new occurrences.

Graceful Degradation

All optional subsystems (face recognition, ANPR, fire/smoke) degrade gracefully. If a model fails to load, the feature reports itself as unavailable and the surveillance pipeline continues unaffected.

13.2 Uniqueness vs Existing Solutions

Unlike traditional CCTV analytics systems that rely on proprietary hardware or cloud services, IBVAP operates entirely on commodity edge hardware with zero external dependencies. The combination of visual night detection, temporal consensus ANPR, Indian plate grammar correction, and SHA-256 hash chain evidence sealing makes it uniquely suited for border surveillance operations where tamper-evidence and offline operation are critical.

Summary

IBVAP delivers a complete, production-ready border surveillance platform that transforms standard CCTV feeds into an AI-powered, tamper-evident analytics system. With sub-10ms rule evaluation, multi-camera support, facial recognition, ANPR, fire/smoke detection, and cryptographically sealed evidence logs, it meets all requirements of SIH 2026 Problem Statement 26187 while operating entirely on commodity edge hardware.